LESSON 14

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Robot Arm

Make a servo motor turn to any angle and steer it with a joystick.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **servo** turns to an exact angle
- How to use a **library** to control a servo
- How a **joystick** is really two knobs and a button
- How to keep a number between limits with `constrain()`

SESSION PLAN

0:00	What's inside a servo?
0:10	Install the **ESP32Servo** library
0:15	Wire the servo, Sketch 1: **L14_Sweep**
0:35	Wire the joystick, Sketch 2: **L14_Joystick**
0:55	Short break
1:00	Sketch 3: **L14_Pointer** (steer the servo)
1:20	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Servo motor SG90 (small blue motor) + a horn (the plastic arm)
- ☐ Joystick module
- ☐ 5 female-to-male jumper wires (for the joystick)
- ☐ Jumper wires

1. Meet the servo

A normal motor just spins round and round. A **servo** is smarter: you tell it an **angle** between 0° and 180°, and it turns to that angle and *holds it there*. Servos move robot arms, steer remote-control cars, and flap the wings on animatronic dinosaurs.

Inside the blue box are a small motor, a stack of gears (to make it strong), a knob that measures where the arm is, and a tiny circuit that keeps correcting the motor until the arm is at the right angle.

> **Big idea: angles in pulses**
>
> The ESP32 tells the servo which angle it wants by sending a pulse 50 times a second. A **short** pulse (0.5 ms) means 0°, a **long** pulse (2.4 ms) means 180°. A **library** does all the timing for us. We just write `myServo.write(90);`

2. Install the library

A **library** is code someone else wrote that we can borrow.

- ☐ **Tools → Manage Libraries…**
- ☐ Search for **ESP32Servo** and click **Install** on *ESP32Servo by Kevin Harrington, John K. Bennett*.

3. Wire the servo

The servo has 3 wires bundled in one plug. Push male jumper wires into the plug's holes.

✓	From	To
☐	ESP32 **GND**	breadboard **–** rail
☐	ESP32 **5V** (may say **VIN**)	breadboard **+** rail
☐	Servo **brown** wire (GND)	**–** rail
☐	Servo **red** wire (power)	**+** rail (5 V)
☐	Servo **orange** wire (signal)	ESP32 pin **18**

Press one of the plastic **horns** onto the servo's shaft so you can see it move.

> **Don't force it!**
>
> Never twist the servo arm by hand when it is powered. You can strip its tiny plastic gears.

4. Sketch 1: Sweep

File → New Sketch, type this in, save as **L14_Sweep**, and upload.

Type this in → save it as **L14_Sweep** — 22 lines

```
// Lesson 14: Servo sweep
// Servo signal (orange wire) on pin 18.

#include <ESP32Servo.h>

const int SERVO_PIN = 18;
Servo myServo;

void setup() {
  myServo.attach(SERVO_PIN, 500, 2400);
}

void loop() {
  for (int angle = 0; angle <= 180; angle++) {
    myServo.write(angle);
    delay(15);
  }
  for (int angle = 180; angle >= 0; angle--) {
    myServo.write(angle);
    delay(15);
  }
}
```

Code	What it means
`#include <ESP32Servo.h>`	Borrow the servo library's code
`Servo myServo;`	Make a servo **object** called `myServo` (like a variable, but it can *do* things)
`myServo.attach(SERVO_PIN, 500, 2400);`	The servo is on pin 18. Pulses go from 500 to 2400 microseconds
`myServo.write(angle);`	Turn to this angle (0–180)
`angle--`	Short for `angle = angle - 1` (and `angle++` adds 1)

Try it

1. Change `delay(15)` to `delay(5)` and to `delay(40)`. What changes?
2. Make it sweep only between 45° and 135°, like a windscreen wiper.

ESP32 restarts or the servo jitters?

The servo can briefly need more power than the USB port gives. Try a different USB port or a powered USB hub. Make sure the servo's brown wire is in **GND**.

5. Wire the joystick

A **joystick** is really **two knobs** (like the potentiometer from Lesson 4) plus a **button** you press by clicking the stick down. One knob measures left–right (**X**), the other up–down (**Y**).

✓	Joystick pin	To
☐	**GND**	**–** rail
☐	**+5V**	ESP32 **3V3** pin (yes, **3V3**, not 5V!)
☐	**VRx**	ESP32 pin **34**
☐	**VRy**	ESP32 pin **35**
☐	**SW**	ESP32 pin **14**

Use 3V3 for the joystick

Even though the joystick's pin says **+5V**, connect it to **3V3**. The ESP32 can only measure voltages up to 3.3 V, and 5 V would damage pins 34 and 35.

6. Sketch 2: Read the joystick

Type this in as **L14_Joystick**, upload, and open the **Serial Monitor**.

Type this in → save it as **L14_Joystick** 27 lines

```
// Lesson 14: Read the joystick
// VRx on 34, VRy on 35, SW (button) on 14.

const int X_PIN = 34;
const int Y_PIN = 35;
const int SW_PIN = 14;

void setup() {
  Serial.begin(115200);
  pinMode(SW_PIN, INPUT_PULLUP);
}

void loop() {
  int x = analogRead(X_PIN);
  int y = analogRead(Y_PIN);
  bool pressed = digitalRead(SW_PIN) == LOW;

  Serial.print("X: ");
  Serial.print(x);
  Serial.print("  Y: ");
  Serial.print(y);
  if (pressed) {
    Serial.print("  CLICK!");
  }
  Serial.println();
  delay(100);
}
```

Map your joystick

Write down the numbers you see:

Stick position	X number	Y number
Middle (let go)		
Full left		
Full right		
Full up		
Full down		

The numbers go from **0** to **4095**. Do you remember from Lesson 4 why? (The ESP32 measures in 4096 tiny steps.) The middle is *about* 1900, but it's never perfectly still. Every joystick wobbles a bit.

7. Sketch 3: Steer the servo

Now let's join them! Pushing the stick one way turns the servo a little bit every 20 ms; pushing it the other way turns it back. Clicking the stick sends the servo back to the middle.

File → **New Sketch**, type this in, and save as **L14_Pointer**.

Type this in → save it as **L14_Pointer** — 36 lines

```
// Lesson 14: Joystick-controlled servo
// Push the stick left/right to turn. Click to go back to the middle.

#include <ESP32Servo.h>

const int SERVO_PIN = 18;
const int X_PIN = 34;
const int SW_PIN = 14;

Servo myServo;
int angle = 90;

void setup() {
  Serial.begin(115200);
  pinMode(SW_PIN, INPUT_PULLUP);
  myServo.attach(SERVO_PIN, 500, 2400);
  myServo.write(angle);
}

void loop() {
  int x = analogRead(X_PIN);

  if (x < 1000) {
    angle = angle + 2;              // stick pushed one way
  } else if (x > 3000) {
    angle = angle - 2;              // stick pushed the other way
  }
  if (digitalRead(SW_PIN) == LOW) {
    angle = 90;
  }

  angle = constrain(angle, 0, 180);
  myServo.write(angle);
  Serial.println(angle);
  delay(20);
}
```

Code	What it means
`int angle = 90;`	Remember where the servo is. Start in the middle
`if (x < 1000)`	The stick is pushed far to one side. Anything between 1000 and 3000 counts as "middle", so small wobbles are ignored
`angle = angle + 2;`	Turn 2° more. Do this 50 times a second and it turns smoothly
`constrain(angle, 0, 180)`	Keep `angle` between 0 and 180. If it's 184, it becomes 180

Turns the wrong way?

Swap `+ 2` and `- 2`. Or turn your joystick around!

8. Challenges

Challenge 1: Speed control

The further you push the stick, the faster the servo turns. *Hint: use* `map()` *to turn the X value into a step size between –5 and +5.*

Challenge 2: Direct drive

Instead of turning step by step, make the servo angle *copy* the stick: `angle = map(x, 0, 4095, 0, 180);` What feels better for a robot arm?

Challenge 3: Robot memory

Click to save the current angle in an **array**. After 5 clicks, make the servo play back all 5 positions in order, 3 times, like a robot repeating a dance. Then use card and tape to turn your servo into a waving robot arm!

9. Quiz

Check what you learned

1. What is the difference between a normal motor and a servo?

   _____

   _____

2. What does `constrain(200, 0, 180)` give back?

   _____

3. Why must the joystick be powered from **3V3** and not 5V?

   _____

   _____

Tidy up: unplug the USB cable. Take the horn off the servo and keep it in the servo's bag.